

Experiment No. 6

Write the Python program to implement BFS (Breadth First Search)

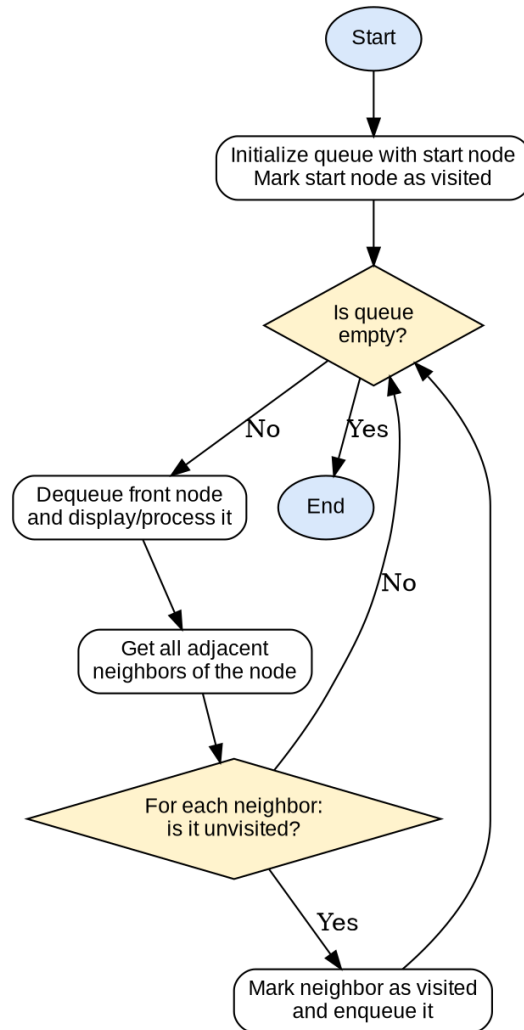
Aim

To write a Python program to implement the Breadth First Search (BFS) algorithm for traversing a graph.

Algorithm

1. Start.
2. Represent the graph using an adjacency list.
3. Initialize an empty queue and an empty visited set; add the start node to both.
4. Repeat while the queue is not empty:
 5. (a) Dequeue the front node and add it to the traversal order.
 6. (b) For every unvisited neighbor of this node, mark it visited and enqueue it.
7. When the queue becomes empty, the traversal order contains all nodes reachable from the start node.
8. Display the BFS traversal order.
9. Stop.

Flowchart



Program

```
from collections import deque
```

```
def bfs(graph, start):
    visited = set()
    queue = deque([start])
    visited.add(start)
    order = []

    while queue:
        node = queue.popleft()
        order.append(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
```

```

        visited.add(neighbor)
        queue.append(neighbor)
    return order

```

```

graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B', 'F'],
    'F': ['C', 'E']
}

```

```

start_node = 'A'
result = bfs(graph, start_node)
print("BFS Traversal starting from node", start_node, ":")
print(" -> ".join(result))

```

Result

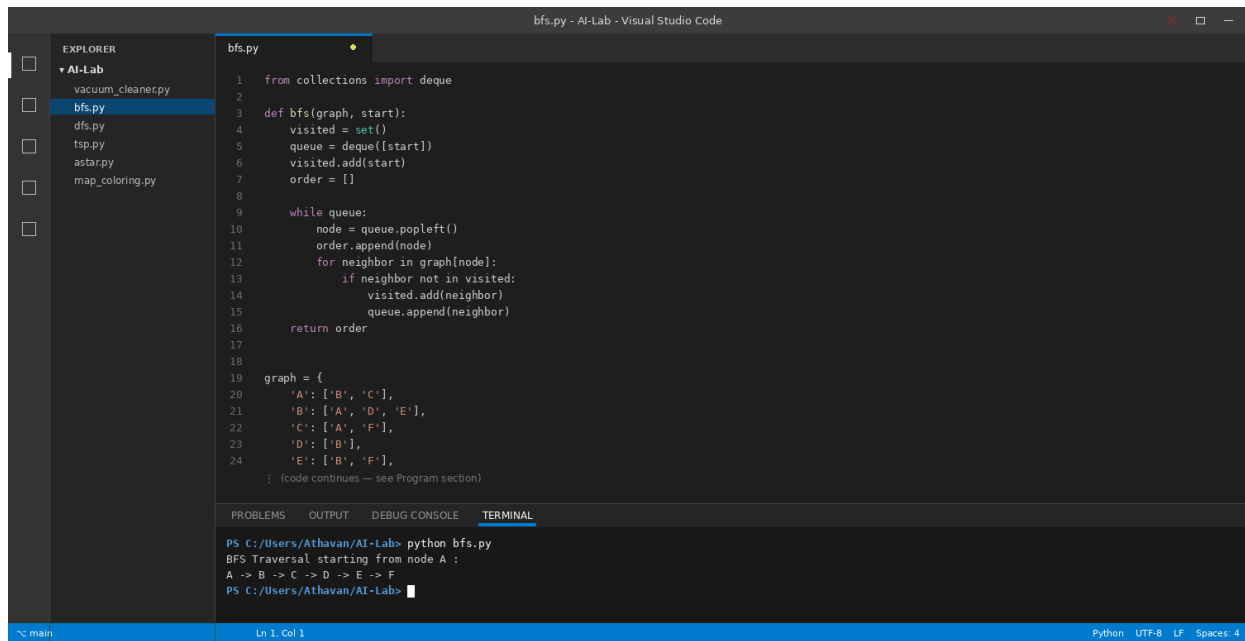
The program was executed successfully and the output was verified as correct.

BFS Traversal starting from node A :

A -> B -> C -> D -> E -> F

Sample Output — Running in VS Code

Screenshot of the program being run in the Visual Studio Code integrated terminal:



The screenshot shows the Visual Studio Code interface with a file explorer on the left, a code editor in the center, and a terminal at the bottom. The file explorer shows a project named 'AI-Lab' with several Python files, including 'bfs.py' which is currently selected. The code editor displays the implementation of a Breadth-First Search (BFS) algorithm. The terminal shows the command 'python bfs.py' being executed, resulting in the output 'BFS Traversal starting from node A : A -> B -> C -> D -> E -> F'.

```
1 from collections import deque
2
3 def bfs(graph, start):
4     visited = set()
5     queue = deque([start])
6     visited.add(start)
7     order = []
8
9     while queue:
10        node = queue.popleft()
11        order.append(node)
12        for neighbor in graph[node]:
13            if neighbor not in visited:
14                visited.add(neighbor)
15                queue.append(neighbor)
16    return order
17
18
19 graph = {
20     'A': ['B', 'C'],
21     'B': ['A', 'D', 'E'],
22     'C': ['A', 'F'],
23     'D': ['B'],
24     'E': ['B', 'F'],
25 }
26
27 : (code continues — see Program section)
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL**

```
PS C:/Users/Athavan/AI-Lab> python bfs.py
BFS Traversal starting from node A :
A -> B -> C -> D -> E -> F
PS C:/Users/Athavan/AI-Lab>
```

Ln 1, Col 1 Python UTF-8 LF Spaces: 4